Project 1

Ioannis Alexandrides (s2767276)

1 Exponential likelihood regression model

1.1 Log-likelihood and score functions

```r
dat <- read.csv("expdata.csv")
y   <- dat$y
x   <- dat$x
cat("n =", length(y), "\n")
```

```
## n = 300
```

```r
summary(dat)
```

```
##        y                   x
##  Min.   :0.0006329   Min.   :0.0006248
##  1st Qu.:0.1355522   1st Qu.:0.2599468
##  Median :0.3058121   Median :0.4795676
##  Mean   :0.4021482   Mean   :0.4996119
##  3rd Qu.:0.5726196   3rd Qu.:0.7328975
##  Max.   :2.4021856   Max.   :0.9942698
```

The model assumes $Y_i \sim \text{Exponential}(\lambda_i)$ where $\lambda_i = \exp(\beta_0 + \beta_1 x_i)$.

Step 1: Log-likelihood. Since the density is $f(y_i \mid \lambda_i) = \lambda_i \exp(-\lambda_i y_i)$, the log-likelihood for n independent observations is

$$\ell(\beta_0, \beta_1) = \sum_{i=1}^{n} \big[\log \lambda_i - \lambda_i y_i\big] = n\beta_0 + \beta_1 \sum_{i=1}^{n} x_i - e^{\beta_0} \sum_{i=1}^{n} y_i e^{\beta_1 x_i}.$$

Step 2: Score functions. Differentiating with respect to each parameter:

$$\frac{\partial \ell}{\partial \beta_0} = n - e^{\beta_0} \sum_{i=1}^{n} y_i e^{\beta_1 x_i}, \qquad \frac{\partial \ell}{\partial \beta_1} = \sum_{i=1}^{n} x_i - e^{\beta_0} \sum_{i=1}^{n} x_i y_i e^{\beta_1 x_i}.$$

Step 3: Closed-form $\hat{\beta}_0(\beta_1)$. Setting the first score to zero and solving:

$$n = e^{\beta_0} \sum_{i=1}^{n} y_i e^{\beta_1 x_i} \implies \hat{\beta}_0(\beta_1) = \log n - \log\left(\sum_{i=1}^{n} y_i e^{\beta_1 x_i}\right).$$

Step 4: Profile log-likelihood. Since the first score equation can be solved explicitly for β_0 as a function of β_1, any joint maximum $(\hat{\beta}_0, \hat{\beta}_1)$ must satisfy this relationship. Substituting $\hat{\beta}_0(\beta_1)$ into $\ell(\beta_0, \beta_1)$ yields the profile log-likelihood

$$\ell_p(\beta_1) = \ell\Big(\hat{\beta}_0(\beta_1),\, \beta_1\Big),$$

which depends only on β_1. Maximising over (β_0, β_1) is therefore equivalent to maximising $\ell_p(\beta_1)$ over β_1 alone. Once $\hat{\beta}_1$ is obtained, $\hat{\beta}_0$ follows immediately without further numerical optimisation.

1.2 MLE via `optim()`

The function `negloglik_expreg(par, y, x)` computes the negative log-likelihood $-\ell(\beta_0, \beta_1)$.
The function `fit_exp_reg(y, x)` then minimises this using `optim()` with the BFGS method and `hessian = TRUE`, so the Hessian matrix is returned at the optimum. The covariance matrix is obtained as

$$\hat{V} = \left[-\nabla^2 \ell(\hat{\beta})\right]^{-1},$$

and the standard errors are the square roots of its diagonal entries.

```r
mle_fit <- fit_exp_reg(y, x)
cat("Estimates:\n");           print(round(mle_fit$estimates, 4))
```

```
## Estimates:

## beta0 beta1
## 0.650 0.545
```

```r
cat("\nHessian:\n");           print(round(mle_fit$hessian,   4))
```

```
##
## Hessian:

##          beta0    beta1
## beta0 299.9999 149.8836
## beta1 149.8836  97.9109
```

```r
cat("\nCovariance matrix:\n"); print(round(mle_fit$vcov,      6))
```

```
##
## Covariance matrix:

##           beta0     beta1
## beta0  0.014173 -0.021697
## beta1 -0.021697  0.043427
```

```r
cat("\nStandard errors:\n");   print(round(mle_fit$se,        4))
```

```
##
## Standard errors:

##  beta0  beta1
## 0.1191 0.2084
```

$$H = \begin{pmatrix} 299.9999 & 149.8836 \\ 149.8836 & 97.9109 \end{pmatrix}$$

$$V = \begin{pmatrix} 0.0142 & -0.0217 \\ -0.0217 & 0.0434 \end{pmatrix}$$

The profile log-likelihood is plotted below to inspect the shape of the objective function in one dimension.

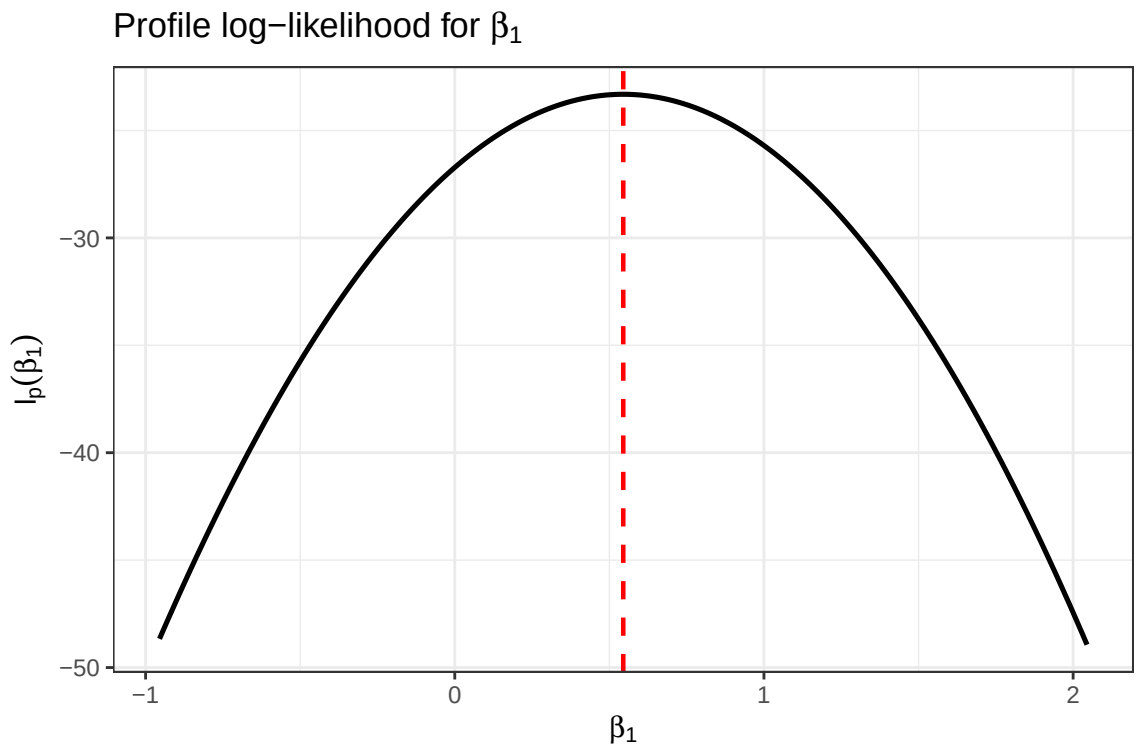

Figure 1: Profile log-likelihood for beta1, with beta0 analytically concentrated out. The red dashed line marks the MLE.

The profile log-likelihood is smooth and strictly concave in a neighbourhood of $\hat{\beta}_1$, indicating that the MLE is unique. Near the peak, the curve is approximately quadratic, which is consistent with the normal approximation to the distribution of the MLE that underlies Wald inference.

1.3 Wald confidence intervals

The function `CI(y, x, alpha)` computes Wald confidence intervals for β_0, β_1, e^{β_1}, and $e^{-\beta_1}$. Used Direct Wald intervals for the regression coefficients. Lastly the delta method is applied for the transformed parameters: for a smooth function $g(\beta)$,

$$\mathrm{Var}\{g(\hat{\beta})\} \approx \nabla g(\hat{\beta})^\top \hat{V} \, \nabla g(\hat{\beta}).$$

For $g(\beta) = e^{\beta_1}$ the gradient is $(0,\, e^{\beta_1})^\top$, so

$$\widehat{\mathrm{se}}(e^{\hat{\beta}_1}) = e^{\hat{\beta}_1} \, \widehat{\mathrm{se}}(\hat{\beta}_1).$$

For $e^{-\beta_1}$ the gradient is $(0, -e^{-\beta_1})^\top$, giving the same magnitude scaled by $e^{-\hat{\beta}_1}$.

```
ci_table <- CI(y, x, alpha = 0.05)
knitr::kable(ci_table, digits = 4,
             caption = "95% Wald confidence intervals")
```

Table 1: 95% Wald confidence intervals

param	hat	se	lo	hi
beta0	0.6500	0.1191	0.4167	0.8833
beta1	0.5450	0.2084	0.1365	0.9534
exp(beta1)	1.7246	0.3594	1.0202	2.4289
exp(-beta1)	0.5799	0.1208	0.3430	0.8167

Interpretation. All three quantities can be interpreted directly in terms of the hazard.

- β_1 represents the log-hazard ratio per unit increase in x. A positive estimate indicates that higher covariate values are associated with a higher event rate.
- e^{β_1} is the hazard ratio itself. It gives the multiplicative change in the hazard when x increases by one unit. If the 95% confidence interval excludes 1, this suggests a statistically significant effect of the covariate.
- $e^{-\beta_1}$ can be regarded as the ratio of mean waiting times. Since $E[Y_i] = \lambda_i^{-1}$, a one-unit increase in x multiplies the expected waiting time by $e^{-\beta_1}$. Values below 1 indicate shorter expected waiting times at higher covariate levels.

1.4 Coverage study

The function `estimate_coverage` estimates the empirical coverage of the 95% Wald confidence interval for β_1 using simulation. For each of $N = 10000$ replicates, n covariate values are resampled with replacement from the observed x, and responses are generated from the exponential regression model at the true parameter values. The Wald CI is then computed and checked to see the coverage of β_1.

The MLE from the full dataset is used as the true parameter vector for the simulation. It evaluates the finite-sample performance at values which are consistent with the observed data.

```
b0_true <- mle_fit$estimates["beta0"]
b1_true <- mle_fit$estimates["beta1"]
ns      <- c(30, 50, 100, 200, 500, 1000)

cov_df <- data.frame(n = ns)
cov_df$coverage <- sapply(ns, function(n_i) {
  estimate_coverage(n_i, b0_true, b1_true, x,
                    N = 10000, seed = 12345L)
})
cov_df$mc_se <- sqrt(cov_df$coverage * (1 - cov_df$coverage) / 10000)
knitr::kable(cov_df, digits = 4,
             caption = "Empirical coverage and Monte Carlo SE")
```

Table 2: Empirical coverage and Monte Carlo SE

n	coverage	mc_se
30	0.9470	0.0022
50	0.9465	0.0023
100	0.9508	0.0022
200	0.9489	0.0022
500	0.9511	0.0022
1000	0.9459	0.0023

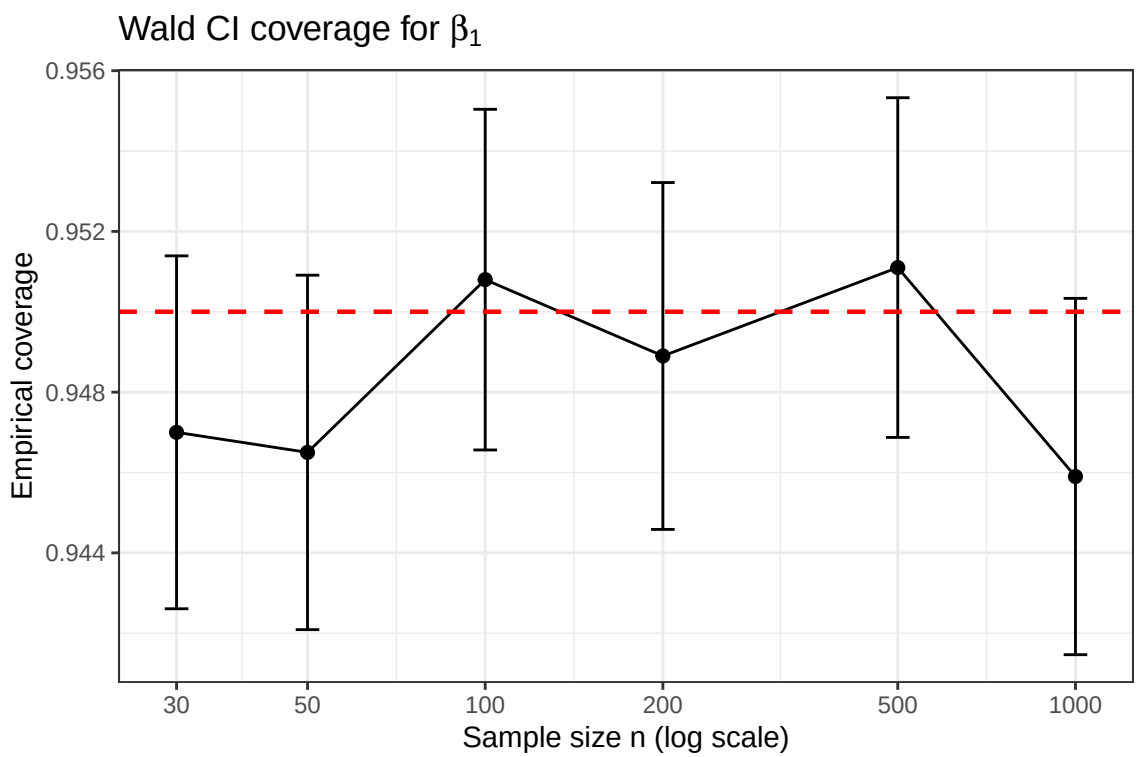

Figure 2: The empirical coverage of the 95% Wald CI for beta1 across sample sizes. The error bars show +/- 1.96 Monte Carlo SE. Lastly the red dashed line marks the nominal 95% level.

Large-sample behaviour. Under standard regularity conditions the MLE satisfies

$$\sqrt{n}(\hat{\beta} - \beta) \xrightarrow{d} N\big(0,\, I(\beta)^{-1}\big)\,,$$

where $I(\beta)$ is the Fisher information matrix. This asymptotic normality in the results justifies the Wald intervals for large n. For smaller sample sizes, the log-likelihood may be somewhat skewed, and the Wald interval — which is symmetric about $\hat{\beta}_1$ — can therefore under- or over-cover. As n increases, the quadratic approximation becomes more accurate and the empirical coverage moves closer to the nominal 95% level, as seen in the simulation results.

Monte Carlo standard error. With $N = 10000$ replicates, the Monte Carlo standard error is at most $\sqrt{0.25/10000} = 0.005$. This means that any differences in coverage across sample sizes are due to genuine finite-sample effects rather than simulation variability.

2 Latent variable Poisson model

2.1 Monte Carlo approximation

2.1.1 Derivation of the MC estimator

The model is $X_i \sim N(0, \sigma^2)$ and $Y_i \mid X_i = x \sim \text{Poisson}(e^{\mu+x})$, where all observations are independent. By the law of total expectation,

$$p(m \mid \mu, \sigma) = E_X[\Pr(Y = m \mid X)] = E_X\left[\texttt{dpois}(m,\, e^{\mu+X})\right].$$

This expectation can be estimated by drawing $X^{(1)}, \ldots, X^{(K)} \overset{\text{iid}}{\sim} N(0, \sigma^2)$ and computing

$$\hat{p}_K(m) = \frac{1}{K} \sum_{k=1}^{K} \texttt{dpois}\left(m,\, e^{\mu+X^{(k)}}\right).$$

Consistency. Each term lies in $[0, 1]$, so the summands are i.i.d. with finite expectation $p(m \mid \mu, \sigma)$. The Strong Law of Large Numbers gives $\hat{p}_K(m) \xrightarrow{\text{a.s.}} p(m \mid \mu, \sigma)$ as $K \to \infty$.

The marginal mean of Y follows from the log-normal moment formula $E[e^X] = e^{\sigma^2/2}$ for $X \sim N(0, \sigma^2)$:

$$E[Y] = E\left[e^{\mu+X}\right] = e^{\mu+\sigma^2/2}.$$

Lastly, The Monte Carlo standard error is estimated as

$$\widehat{\text{se}}(m) = \frac{\text{sd}(Z^{(1)}, \ldots, Z^{(K)})}{\sqrt{K}}$$

where $Z^{(k)} = \texttt{dpois}(m, e^{\mu+X^{(k)}})$, following directly from the standard error of a sample mean.

2.1.2 PMF estimation and comparison with Poisson

```r
mu_val    <- 0.2
sigma_val <- 0.8
m_vals    <- 0:20

pmf_mc <- dlatentpois_mc(m_vals, mu = mu_val, sigma = sigma_val, K = 10000)
mean_Y <- exp(mu_val + sigma_val^2 / 2)
cat("Marginal mean E[Y] =", round(mean_Y, 4), "\n")
```

```
## Marginal mean E[Y] = 1.682
```

```r
head(pmf_mc)
```

```
##   m        pmf           se
## 1 0 0.31846044 0.0022349295
## 2 1 0.27632720 0.0009958215
## 3 2 0.17209930 0.0007929497
## 4 3 0.09814462 0.0007465808
## 5 4 0.05521812 0.0006180820
## 6 5 0.03143828 0.0004804696
```

The marginal variance is derived by using the law of total variance:

$$\mathrm{Var}(Y) = E[\mathrm{Var}(Y \mid X)] + \mathrm{Var}(E[Y \mid X]) = E[\lambda(X)] + \mathrm{Var}(\lambda(X)).$$

Since $\lambda(X) = e^{\mu+X}$ is log-normally distributed,

$$\mathrm{Var}(Y) = e^{\mu+\sigma^2/2} + e^{2\mu+\sigma^2}(e^{\sigma^2} - 1),$$

which exceeds the mean for any $\sigma^2 > 0$. The model therefore displays overdispersion relative to a Poisson distribution with the same mean.

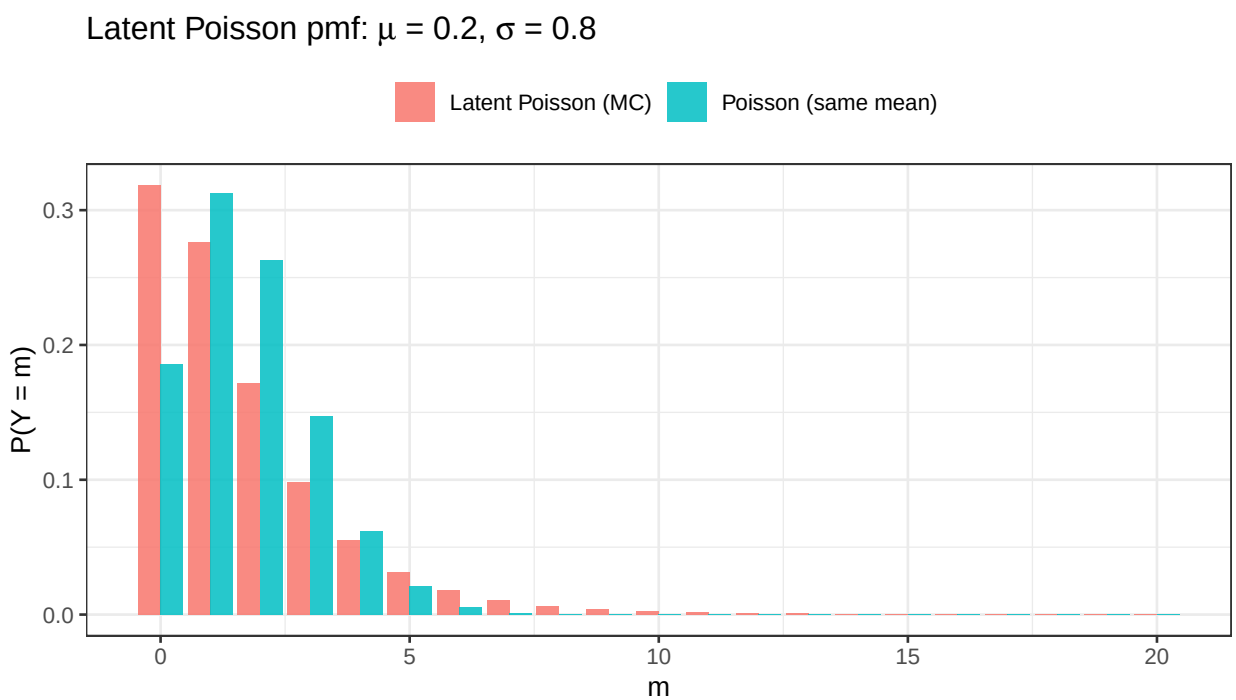

Figure 3: Marginal pmf of the latent Poisson model (K = 10000) compared with a Poisson having the same mean.

The plot confirms that the latent Poisson assigns more probability to extreme counts in both tails, on both small and large counts, compared to the simple Poisson. This reflects the overdispersion induced by the latent Gaussian variability. This direct consequence of the overdispersion shown analytically above, illustrates why a Poisson model would be misspecified for data generated by this hierarchical structure.

2.1.3 Monte Carlo standard errors for different K

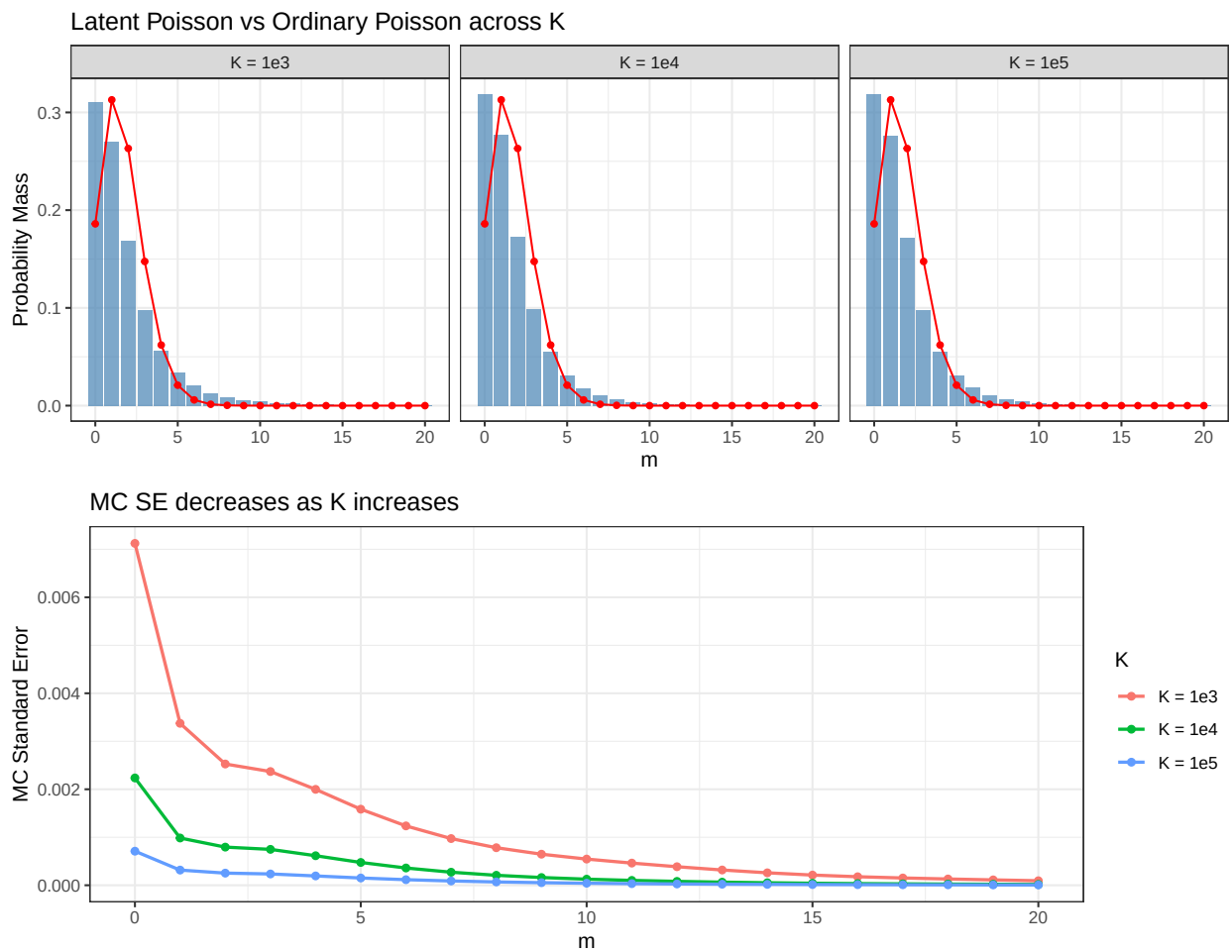

Figure 4: Top: Latent Poisson pmf vs Ordinary Poisson across K. Bottom: MC standard errors across K. Both confirm convergence as K increases.

The Monte Carlo standard error decreases at the expected rate $1/\sqrt{K}$, so increasing K by a factor of 10 reduces the error by $\sqrt{10} \approx 3.16$, in line with the Central Limit Theorem. The SE peaks near the mode because the Poisson probability is the largest and thus the most variable there across Monte Carlo draws. SE is smallest in the tails where most draws contribute near-zero values.

2.2 Importance sampling approximation

2.2.1 Importance sampling estimator

The marginal pmf is an expectation under $X \sim N(0, \sigma^2)$. Using the proposal $q(x) = N(a, \sigma^2)$, the importance weights are

$$w(x) = \frac{\phi(x;\, 0,\, \sigma^2)}{\phi(x;\, a,\, \sigma^2)},$$

and the importance sampling estimator is

$$\hat{p}^{\mathrm{IS}}(m) = \frac{1}{K}\sum_{k=1}^{K} \texttt{dpois}\Big(m,\, e^{\mu+X^{(k)}}\Big)\, w(X^{(k)}), \quad X^{(k)} \sim N(a, \sigma^2).$$

The weights are computed on the log scale for numerical stability, thus avoiding underflow when evaluating normal densities far from zero. The function `dlatentpois_is(m, mu, sigma, a, K)` implements this estimator.

The Monte Carlo standard error is

$$\widehat{\mathrm{se}}(m) = \frac{\mathrm{sd}(Z^{(1)}, \ldots, Z^{(K)})}{\sqrt{K}}$$

where $Z^{(k)} = \texttt{dpois}(m, e^{\mu+X^{(k)}})\, w(X^{(k)})$.

2.2.2 Comparison with crude MC for m = 10

```
K_val    <- 5000
a_values <- c(-1, 0, 1, log(10) - mu_val)

comparison_df <- do.call(rbind, lapply(a_values, function(a_i) {
  res <- dlatentpois_is(10, mu = mu_val, sigma = sigma_val,
                        a = a_i, K = K_val)
  res$a <- round(a_i, 3)
  res
}))
print(comparison_df, digits = 6)
```

```
##    m        pmf          se      a
## 1 10 0.00323894 1.05041e-03 -1.000
## 2 10 0.00297881 2.02885e-04  0.000
## 3 10 0.00276761 5.33454e-05  1.000
## 4 10 0.00280808 3.85731e-05  2.103
```

```
cat("\nCrude MC (a=0) SE:          ", round(comparison_df$se[2], 6))
```

```
##
## Crude MC (a=0) SE:           0.000203
```

```
cat("\nOptimal IS SE (a=log10-mu): ", round(comparison_df$se[4], 6))
```

```
##
## Optimal IS SE (a=log10-mu):  3.9e-05
```

```
cat("\nVariance reduction factor:  ",
    round((comparison_df$se[2] / comparison_df$se[4])^2, 2), "\n")
```

```
##
## Variance reduction factor:   27.66
```

The table reports results for four choices of the proposal mean a. When $a = 0$, the proposal distribution coincides with the target, so the method reduces to crude Monte Carlo. As a increases toward $\log(10) - \mu$, the proposal centres on the region where `dpois`$(10, e^{\mu+x})$ is largest, and the Monte Carlo SE falls substantially. The value $a = \log(10) - \mu$ is optimal because it aligns the proposal mean with the value of x at which the Poisson rate equals $m = 10$, maximising overlap between the proposal and the integrand. The point estimates are stable across all values of a, confirming that the estimator remains unbiased regardless of the proposal. The variance reduction factor summarises the efficiency gain compared with crude Monte Carlo.

3 Session information

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Sequoia 15.3.2
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versi
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK ve
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] gridExtra_2.3   lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0
##  [5] dplyr_1.2.0     purrr_1.2.1     readr_2.2.0     tidyr_1.3.2
##  [9] tibble_3.3.1    ggplot2_4.0.2   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] gtable_0.3.6       compiler_4.5.2     tidyselect_1.2.1   scales_1.4.0
##  [5] yaml_2.3.12        fastmap_1.2.0      R6_2.6.1           labeling_0.4.3
##  [9] generics_0.1.4     knitr_1.51         pillar_1.11.1      RColorBrewer_1.1-3
## [13] tzdb_0.5.0         rlang_1.1.7        stringi_1.8.7      xfun_0.56
## [17] S7_0.2.1           timechange_0.4.0   cli_3.6.5          withr_3.0.2
## [21] magrittr_2.0.4     digest_0.6.39      grid_4.5.2         rstudioapi_0.18.0
## [25] hms_1.1.4          lifecycle_1.0.5    vctrs_0.7.1        evaluate_1.0.5
## [29] glue_1.8.0         farver_2.1.2       rmarkdown_2.30     tools_4.5.2
## [33] pkgconfig_2.0.3    htmltools_0.5.9
```

4 Code appendix

```
# Ioannis Alexandrides, s2767276
# Add all your function definitions to this file.
# Each function should be documented using the same style as below.
```

```r
# Exercise 1: Exponential likelihood regression model


# 1(a) & 1(b): Negative log-likelihood and MLE

#' Negative log-likelihood for exponential regression
#'
#' Computes the negative log-likelihood for the exponential regression model
#' y_i ~ Exponential(rate = exp(beta0 + beta1 * x_i)).
#' We return the negative because optim() minimises by default.
#'
#' @param par numeric vector of length 2: c(beta0, beta1)
#' @param y   numeric vector of strictly positive responses
#' @param x   numeric vector of covariates (same length as y)
#' @return numeric scalar: the negative log-likelihood value
negloglik_expreg <- function(par, y, x) {
  beta0  <- par[1]
  beta1  <- par[2]
  lambda <- exp(beta0 + beta1 * x)
  -sum(dexp(y, rate = lambda, log = TRUE))
}

#' Fit exponential regression model by maximum likelihood
#'
#' Minimises the negative log-likelihood using optim() with BFGS.
#' The covariance matrix is the inverse of the observed Hessian at the MLE,
#' and standard errors are the square roots of its diagonal entries.
#'
#' @param y numeric vector of strictly positive responses
#' @param x numeric vector of covariates (same length as y)
#' @return list with components: estimates, hessian, vcov, se
fit_exp_reg <- function(y, x) {
  # Initialise beta0 using log(1/ybar) = -log(ybar), beta1 = 0
  init <- c(beta0 = -log(mean(y)), beta1 = 0)
  fit <- optim(
    par     = init,
    fn      = negloglik_expreg,
    y       = y,
    x       = x,
    method  = "BFGS",
    hessian = TRUE
  )
  vcov_mat  <- solve(fit$hessian)
  se        <- sqrt(diag(vcov_mat))
  names(se) <- c("beta0", "beta1")
  list(
    estimates = setNames(fit$par, c("beta0", "beta1")),
    hessian   = fit$hessian,
    vcov      = vcov_mat,
    se        = se
  )
}
```

```r
# 1(b): Profile log-likelihood (used for the plot in part b)

#' Profile estimate of beta0 as a function of beta1
#'
#' Closed-form solution from setting the first score equation to zero.
#' This lets us reduce the 2D optimisation to 1D.
#'
#' @param beta1 numeric scalar
#' @param y     numeric vector of strictly positive responses
#' @param x     numeric vector of covariates (same length as y)
#' @return numeric scalar: profile MLE of beta0
beta0_profile <- function(beta1, y, x) {
  log(length(y)) - log(sum(y * exp(beta1 * x)))
}

#' Profile log-likelihood as a function of beta1
#'
#' Substitutes the closed-form beta0_profile(beta1, y, x) into the
#' log-likelihood, giving a function of beta1 alone.
#'
#' @param beta1 numeric scalar
#' @param y     numeric vector of strictly positive responses
#' @param x     numeric vector of covariates (same length as y)
#' @return numeric scalar: profile log-likelihood value
loglik_profile <- function(beta1, y, x) {
  b0 <- beta0_profile(beta1, y, x)
  -negloglik_expreg(c(b0, beta1), y, x)
}

# 1(c): Wald confidence intervals

#' Wald confidence intervals for exponential regression parameters
#'
#' Computes Wald CIs for beta0, beta1, exp(beta1) and exp(-beta1).
#' Direct Wald intervals for the regression coefficients; delta method
#' for the two transformations.
#'
#' @param y     numeric vector of strictly positive responses
#' @param x     numeric vector of covariates (same length as y)
#' @param alpha numeric significance level, default 0.05
#' @return data frame with columns: param, hat, se, lo, hi
CI <- function(y, x, alpha = 0.05) {
  mle <- fit_exp_reg(y, x)
  b   <- mle$estimates
  V   <- mle$vcov
  z   <- qnorm(1 - alpha / 2)

  # Wald CIs for beta0 and beta1
  rows <- data.frame(
    param = c("beta0", "beta1"),
    hat   = as.numeric(b),
    se    = as.numeric(mle$se),
    lo    = as.numeric(b) - z * as.numeric(mle$se),
```

```r
    hi    = as.numeric(b) + z * as.numeric(mle$se),
    row.names = NULL
  )

  # Delta method for exp(beta1): gradient is (0, exp(beta1))
  eb1    <- exp(b["beta1"])
  g1     <- c(0, eb1)
  se_eb1 <- sqrt(as.numeric(t(g1) %*% V %*% g1))
  rows   <- rbind(rows, data.frame(
    param = "exp(beta1)",
    hat   = as.numeric(eb1),
    se    = se_eb1,
    lo    = as.numeric(eb1) - z * se_eb1,
    hi    = as.numeric(eb1) + z * se_eb1
  ))

  # Delta method for exp(-beta1): gradient is (0, -exp(-beta1))
  emb1    <- exp(-b["beta1"])
  g2      <- c(0, -emb1)
  se_emb1 <- sqrt(as.numeric(t(g2) %*% V %*% g2))
  rows    <- rbind(rows, data.frame(
    param = "exp(-beta1)",
    hat   = as.numeric(emb1),
    se    = se_emb1,
    lo    = as.numeric(emb1) - z * se_emb1,
    hi    = as.numeric(emb1) + z * se_emb1
  ))

  rows
}

# 1(d): Coverage simulation

#' Empirical coverage probability of the Wald CI for beta1
#'
#' Simulates N datasets of size n, fits the model to each, and checks
#' whether the Wald CI for beta1 covers the true value. Covariates are
#' resampled with replacement from the observed x vector.
#'
#' @param n     integer sample size for each simulated dataset
#' @param beta0 numeric true intercept
#' @param beta1 numeric true slope
#' @param x     numeric covariate pool to resample from
#' @param N     integer number of simulation replicates, default 10000
#' @param alpha numeric significance level for CI, default 0.05
#' @param seed  integer optional random seed for reproducibility
#' @return numeric scalar: empirical coverage probability
estimate_coverage <- function(n, beta0, beta1, x, N = 10000,
                              alpha = 0.05, seed = NULL) {
  if (!is.null(seed)) set.seed(seed)
  z       <- qnorm(1 - alpha / 2)
  covered <- logical(N)
  for (i in seq_len(N)) {
```

```r
    xi     <- sample(x, size = n, replace = TRUE)
    lambda <- exp(beta0 + beta1 * xi)
    yi     <- rexp(n, rate = lambda)
    # tryCatch handles rare numerical failures gracefully
    fit    <- tryCatch(fit_exp_reg(yi, xi), error = function(e) NULL)
    if (is.null(fit)) next
    b1_hat     <- fit$estimates["beta1"]
    se_b1      <- fit$se["beta1"]
    covered[i] <- (b1_hat - z * se_b1 <= beta1) &&
      (beta1  <= b1_hat + z * se_b1)
  }
  mean(covered)
}


# Exercise 2: Latent variable Poisson model


# 2.1(a) & 2.1(b): Monte Carlo estimator

#' Monte Carlo estimator of the marginal pmf of the latent Poisson model
#'
#' Model: X ~ N(0, sigma^2), Y|X ~ Poisson(exp(mu + X)).
#' Estimates p(m | mu, sigma) by averaging dpois(m, exp(mu + X)) over
#' K draws from the latent distribution. Consistent by the SLLN.
#'
#' @param m     integer vector of count values to evaluate the pmf at
#' @param mu    numeric scalar mean parameter
#' @param sigma numeric scalar standard deviation, sigma > 0
#' @param K     integer number of Monte Carlo samples
#' @return data frame with columns: m, pmf, se
dlatentpois_mc <- function(m, mu, sigma, K) {
  X   <- rnorm(K, mean = 0, sd = sigma)
  lam <- exp(mu + X)
  out <- lapply(m, function(m_j) {
    h <- dpois(m_j, lambda = lam)
    data.frame(m = m_j, pmf = mean(h), se = sd(h) / sqrt(K))
  })
  do.call(rbind, out)
}

# 2.2(a) & 2.2(b): Importance sampling estimator

#' Importance sampling estimator of the marginal pmf of the latent Poisson model
#'
#' Uses proposal q(x) = N(a, sigma^2) instead of the target N(0, sigma^2).
#' Importance weights w(x) = phi(x; 0, sigma^2) / phi(x; a, sigma^2) correct
#' for the distributional shift. Weights computed on the log scale to avoid
#' numerical underflow for large shifts in a.
#'
#' @param m     integer vector of count values to evaluate the pmf at
#' @param mu    numeric scalar mean parameter
#' @param sigma numeric scalar standard deviation, sigma > 0
```

```r
#' @param a     numeric scalar proposal mean
#' @param K     integer number of importance samples
#' @return data frame with columns: m, pmf, se
dlatentpois_is <- function(m, mu, sigma, a, K) {
  X_q   <- rnorm(K, mean = a, sd = sigma)
  # Log-scale weights to avoid underflow
  log_w <- dnorm(X_q, mean = 0, sd = sigma, log = TRUE) -
    dnorm(X_q, mean = a, sd = sigma, log = TRUE)
  w     <- exp(log_w)
  lam   <- exp(mu + X_q)
  out <- lapply(m, function(m_j) {
    hw <- dpois(m_j, lambda = lam) * w
    data.frame(m = m_j, pmf = mean(hw), se = sd(hw) / sqrt(K))
  })
  do.call(rbind, out)
}
```